ENGR 1451/2451: Exploratory Data Science

Fall 2026 Syllabus Overview

Prof. Paul W. Leu
University of Pittsburgh

Fall 2026

Course at a Glance

Courses	ENGR 1451 (undergraduate) and ENGR 2451 (graduate)
Instructor	Prof. Paul W. Leu, pleu@pitt.edu
Semester	Fall 2026
Class location	312 Benedum Hall
Instructor office hours	Tuesdays, 11:00–11:30 AM, 218 F Benedum Hall
TA	Boyi Sun, BOS69@pitt.edu
Course site	University of Pittsburgh Canvas

Why Data Analytics?

Data is changing how engineers design, manufacture, optimize, and make decisions.

- Companies have strong demand for engineers who can analyze data and communicate results.
- Data analytics skills complement every engineering major.

Goal: combine engineering domain knowledge with modern data methods.

Skills in Demand

Core technical skills

- Program in R or Python
- Analyze data with statistics
- Build and interpret models
- Use machine learning and AI tools
- Make clear visualizations

Engineering applications

- Make better decisions
- Optimize supply chains
- Manufacture products
- Manage energy systems
- Develop new applications

Career Motivation

Projected growth	Salary signal
34%	$166K
Projected growth in data-scientist jobs from 2024–2034[1]	Average U.S. data-scientist salary in Q1 2026[2]

[1] U.S. Bureau of Labor Statistics, Occupational Outlook Handbook: Data Scientists.

[2] 365 Data Science, Data Scientist Job Outlook 2026 (Glassdoor Q1 2026 data).

Engineering Data Analytics Certificate

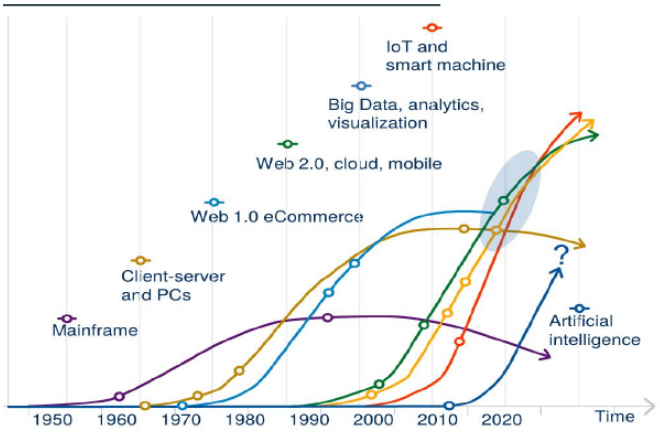
The certificate is a way to show employers that your engineering degree includes formal data-analytics training.

The Engineering Data Analytics Certificate requires **15 credits** organized around three domains.

Domain	Purpose
1. Foundations	Programming and inferential statistics
2. Expertise	Exploratory/applied data science plus modeling and prediction
3. Specialization	A semester-long engineering data analytics project

- At least **9 credits** must be beyond courses required for your major.
- Students may count a maximum of **two major courses** toward the certificate.

Digital Transformation: Combinatorial Effects Accelerate Change



Cheryl Martin, Jim Hageman Snabe, and Pierre Nanterme, *Introducing the Digital Transformation Initiative*, World Economic Forum, 2018.

Falling Technology Costs Accelerate Digital Transformation

Falling cost of advanced technologies

- A defining characteristic of the digital revolution
- Computing
- Internet communications
- Data storage

Drones

2007: **$100,000** per unit

2013: **$700** per unit

DNA sequencing

2000: **$2.7 billion**

2014: **$1,000**

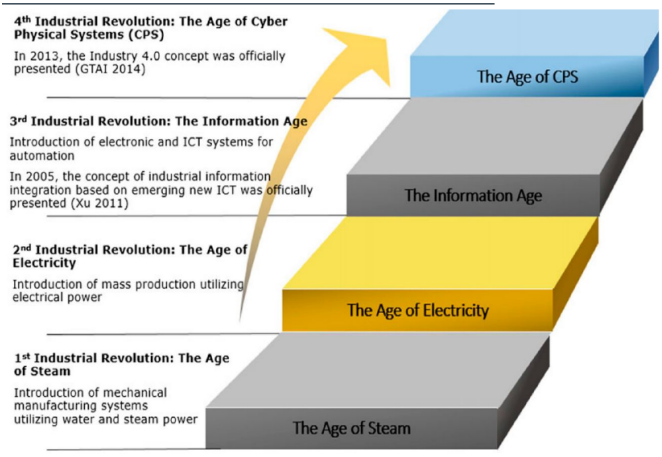
Solar

1984: **$30/kWh**

2014: **$0.16/kWh**

Cheryl Martin, Jim Hageman Snabe, and Pierre Nanterme, *Introducing the Digital Transformation Initiative*, World Economic Forum, 2018.

Industry 4.0: The 4th Industrial Revolution



Y. Lu, "Industry 4.0: A survey on technologies, applications and open research issues," *Journal of Industrial Information Integration*, 6 (2017), 1–10.

https://doi.org/10.1016/j.jii.2017.04.005

What Is Exploratory Data Science?

Exploratory data science combines computational tools, statistical reasoning, visualization, and domain knowledge to understand real-world data.

- Assemble, clean, explore, visualize, and interpret datasets.

What Is Exploratory Data Science?

Exploratory data science combines computational tools, statistical reasoning, visualization, and domain knowledge to understand real-world data.

- Assemble, clean, explore, visualize, and interpret datasets.
- Identify statistically meaningful relationships.

What Is Exploratory Data Science?

Exploratory data science combines computational tools, statistical reasoning, visualization, and domain knowledge to understand real-world data.

- Assemble, clean, explore, visualize, and interpret datasets.
- Identify statistically meaningful relationships.
- Develop initial predictive models.

What Is Exploratory Data Science?

Exploratory data science combines computational tools, statistical reasoning, visualization, and domain knowledge to understand real-world data.

- Assemble, clean, explore, visualize, and interpret datasets.
- Identify statistically meaningful relationships.
- Develop initial predictive models.
- Emphasize interpretation rather than only numerical output.

“Structure of a Data Analysis” Perspective

1. Define the question

Perspective attributed in the source deck to Jeff Leek, Johns Hopkins University, “Data Analytic Style.”

“Structure of a Data Analysis” Perspective

1. Define the question
2. Clean and explore the data

Perspective attributed in the source deck to Jeff Leek, Johns Hopkins University, “Data Analytic Style.”

“Structure of a Data Analysis” Perspective

1. Define the question
2. Clean and explore the data
3. Model, predict, and learn

Perspective attributed in the source deck to Jeff Leek, Johns Hopkins University, “Data Analytic Style.”

“Structure of a Data Analysis” Perspective

1. Define the question
2. Clean and explore the data
3. Model, predict, and learn
4. Present final models and learnings

Perspective attributed in the source deck to Jeff Leek, Johns Hopkins University, “Data Analytic Style.”

“Structure of a Data Analysis” Perspective

1: Define the question

- Establish background and critical issues.
- Define the question and ideal dataset.
- Determine accessible data and required capabilities/packages.
- Obtain, structure, clean, and tidy the data.

Perspective attributed in the source deck to Jeff Leek, Johns Hopkins University, “Data Analytic Style.”

“Structure of a Data Analysis” Perspective

1: Define the question

- Establish background and critical issues.
- Define the question and ideal dataset.
- Determine accessible data and required capabilities/packages.
- Obtain, structure, clean, and tidy the data.

2: Cleaning and exploratory data analysis

- Write a databook defining variables, units, and structures.
- Visualize and explore trends and functional forms.
- Consider power transformations.
- Validate against domain knowledge.
- Select appropriate modeling approaches.

Perspective attributed in the source deck to Jeff Leek, Johns Hopkins University, “Data Analytic Style.”

“Structure of a Data Analysis” Perspective

3: Modeling, prediction, machine learning

- Try statistical and machine-learning models.
- Perform model selection and cross-validation.
- Evaluate predictive R^2.
- Interpret the results.
- Challenge the results.

“Structure of a Data Analysis” Perspective

3: Modeling, prediction, machine learning

- Try statistical and machine-learning models.
- Perform model selection and cross-validation.
- Evaluate predictive R^2.
- Interpret the results.
- Challenge the results.

4: Present final models and learnings

- Present results.
- Present reproducible code.
- Compare with modeling approaches in the literature.

“Structure of a Data Analysis” Perspective

3: Modeling, prediction, machine learning

- Try statistical and machine-learning models.
- Perform model selection and cross-validation.
- Evaluate predictive R^2.
- Interpret the results.
- Challenge the results.

4: Present final models and learnings

- Present results.
- Present reproducible code.
- Compare with modeling approaches in the literature.

Software packaging/engineering

Scripts $\Rightarrow$ functions $\Rightarrow$ packages

Documentation and vignettes

Publication on CRAN or PyPI

Course Emphasis

- Python is the primary programming language.
- Examples will draw from engineering and scientific datasets.
- Datasets may include time-series, spectral, image-derived, and multivariable data.
- ENGR 1451 and ENGR 2451 meet together.
- ENGR 2451 students complete additional work through a semester-long data-science project.

Why Python?

Why Python for this course?

- General-purpose language for data, modeling, automation, and software
- Strong engineering data stack: NumPy, pandas, matplotlib, scikit-learn
- Widely used in machine learning and AI: PyTorch, JAX, TensorFlow
- Works well with GitHub, Jupyter/Colab, cloud computing, and open-source tools

Other options

- **R:** excellent for statistics and visualization, but less general for engineering workflows
- **MATLAB:** strong for numerical computing and controls, but commercial and less open

Python and Data-Science Tools

Core environment

- Python 3
- JupyterLab or Jupyter Notebook
- VS Code with a Python environment
- Canvas for course materials and submissions

Scientific Python stack

- NumPy and pandas
- Matplotlib and related visualization tools
- SciPy and statsmodels
- scikit-learn
- Git/GitHub where appropriate

Course Learning Outcomes I

By the end of the course, students should be able to:

- Use Python to import, inspect, clean, transform, and organize data.
- Use NumPy and pandas for numerical and tabular data analysis.
- Create clear and informative data visualizations using Python.
- Compute and interpret descriptive statistics and probability distributions.
- Apply exploratory data analysis to identify structure, relationships, outliers, and data-quality problems.
- Formulate and interpret statistical hypotheses and common inferential tests.

Course Learning Outcomes II

- Build and interpret linear, multiple-linear, and logistic regression models.
- Distinguish between explanatory, inferential, and predictive uses of models.
- Evaluate models using appropriate training/testing and diagnostic approaches.
- Apply domain knowledge to determine whether a result is meaningful and defensible.
- Use reproducible workflows with Jupyter notebooks, scripts, and version control.
- Communicate data-science methods, assumptions, results, and limitations clearly in writing.

Prerequisites

Students should have completed:

- an introductory probability/statistics course.

Students who are uncertain about their preparation should contact the instructor early in the semester.

Course Format

Foundation

Statistical concepts, data-science principles, interpretation, and discussion.

Practicum

Python demonstrations, coding exercises, visualization, and analysis of real datasets.

The goal is not only to execute Python code, but also to understand why an analysis is appropriate, what assumptions it makes, and how its results should be interpreted.

Readings and Course Materials

Course notes, example notebooks, datasets, and required readings will be distributed through Canvas.

Useful references include:

- James, Witten, Hastie, Tibshirani, and Taylor, *An Introduction to Statistical Learning: with Applications in Python*.
- OpenIntro, *OpenIntro Statistics*.
- Jake VanderPlas, *Python Data Science Handbook*.
- Python, NumPy, pandas, Matplotlib, SciPy, statsmodels, and scikit-learn documentation.

Assessment Philosophy

Assessment emphasizes both computational practice and individual written understanding.

- Lab exercises use Python for applied data analysis.
- In-class quizzes provide short checks of ongoing understanding.
- The midterm and final are written assessments.
- Quizzes and written exams are completed without a Python environment.
- Code alone is not a complete analysis; interpretation matters.

Lab Exercises

Lab exercises require students to apply Python and statistical methods to real or realistic datasets.

A Claude tutor has been created for the class.

- When working on the lab exercises, you must use this tutor.
- At the end of each session, you must type “make my record" and paste the block it produces into the cell below.

Grading:

- Assignments are not graded for correctness or for how much help you needed,
- Credit is given for an assignment that shows you attempted the problem.

Records are read to find out which steps are hardest, and used to improve the course.

In-class Quizzes

In-class quizzes provide short, low-stakes checks of understanding throughout the semester.

- May include concepts, statistical reasoning, calculations, and interpretation of figures or output.
- May ask students to read brief Python code or pseudocode.
- Generally completed without electronic devices, internet access, generative-AI tools, or a Python environment.
- **Lowest quiz score on any question you attempt is 70. You can retake for half credit**

Written Midterm and Final Exams

Written exams may include:

- short-answer conceptual questions;
- interpretation of figures, tables, statistical output, and model results;
- calculations and statistical reasoning;
- selection and justification of appropriate analysis methods;
- reading, explaining, or debugging short Python code snippets;
- writing short Python statements, algorithms, or pseudocode when appropriate.

Written Exam Rules

Students should focus on understanding the logic of an analysis rather than memorizing large amounts of Python syntax.

Unless explicitly authorized by the instructor or required by an approved accommodation, written exams do **not** permit:

- electronic devices;
- internet access;
- generative-AI tools; or
- coding environments.

ENGR 1451 Grading

Lab Exercises	In-class Quizzes	Written Midterm	Written Final
15%	30%	25%	30%

ENGR 1451 total: 100%

ENGR 2451 Grading

ENGR 2451 is graded on a **140-point basis**.

Shared course assessments: 100 points	
Lab Exercises	**15**
In-class Quizzes	**30**
Written Midterm Exam	**25**
Written Final Exam	**30**

Graduate semester project: 40 points	
Six written semester-project updates	**15**
Final semester project submission	**25**

Total: 140 points

ENGR 2451 Semester Project

ENGR 2451 students complete a semester-long exploratory data-science project related to an engineering, scientific, or research problem.

The project includes:

- a clearly defined question or objective;
- an appropriately documented dataset;
- data cleaning and exploratory analysis;
- visualization and statistical/modeling methods;
- interpretation of results and limitations;
- reproducible Python code; and
- a polished final written report.

Generative AI and Collaboration

- Collaboration is encouraged for learning, debugging, and discussion of lab exercises unless an assignment states otherwise.
- Each student must submit their own work and must understand and explain the code, analysis, and conclusions they submit.
- Generative-AI tools may be used on lab exercises through the Claude tutor.
- Students remain responsible for verifying all generated code and claims.
- Generative-AI tools are not permitted on in-class quizzes, the written midterm, or the written final exam unless explicitly authorized.

Tentative Schedule: Weeks 1–5

Week	Foundation	Python Practicum / Applications
1	Introduction to exploratory data science and reproducibility	Python/Jupyter workflow; Git basics; loading and inspecting data
2	Data types, variables, and data organization	Python fundamentals; NumPy; pandas Series and DataFrames
3	Data quality and exploratory analysis	Importing, cleaning, indexing, filtering, missing data, transformations
4	Summarizing and visualizing data	Descriptive statistics; distributions; Matplotlib; exploratory graphics
5	Random variables and probability distributions	Simulation; sampling; empirical distributions; engineering examples

Tentative Schedule: Weeks 6–10

Week	Foundation	Python Practicum / Applications
6	Relationships among variables	Correlation, covariance, pair plots, multivariable exploration
7	Foundations of statistical inference	Sampling distributions, confidence intervals, hypothesis testing; midterm review
8	**Written Midterm Exam**; reusable analysis workflows	Functions, modular code, notebook organization, reproducibility
9	Categorical data and contingency tables	Proportions, chi-square methods, categorical visualization
10	Numerical inference	One- and two-sample tests; effect size; uncertainty; SciPy/statsmodels

Tentative Schedule: Weeks 11–15

Week	Foundation	Python Practicum / Applications
11	Statistical power and study design	Sample size, practical significance, interpretation of uncertainty
12	Linear regression	Fitting, interpreting, diagnosing regression models; train/test concepts
13	Multiple regression and model selection	Multiple predictors, interactions, collinearity, variable selection, diagnostics
14	Logistic regression and classification	Classification metrics; logistic regression; scikit-learn workflow
15	Statistical learning overview and synthesis	Model comparison; reproducible analysis; final exam review

Course Policies I

Attendance and Participation

Regular attendance and participation are expected. Class meetings include discussion, examples, and practicum work that may not be fully reproduced in posted materials.

Communication

Canvas is the primary source for announcements, due dates, course files, and schedule changes. Email is preferred for individual questions that should not be discussed publicly.

Course Policies II

Academic Integrity

Students are expected to follow the University of Pittsburgh's standards for academic integrity. Unauthorized assistance on written examinations, including use of generative-AI tools, is prohibited.

Accessibility Accommodations

Students who require accommodations should work with Disability Resources and Services and notify the instructor as early as practical.

Changes to the Syllabus

This syllabus describes the planned structure of the course.

The instructor may make reasonable changes to topics, deadlines, or procedures during the semester.

Any substantive changes will be announced through Canvas.

Questions?

Introductions